

Robot Learning: From Fundamentals to Foundation Models

Lecture 6: Generative Models

Oier Mees

ETH zürich



Microsoft

23.03.2026

Recap: Why Generative Modeling for Robotics?

Stochasticity: Many (**infinite**) ways to close a drawer with a 7-DoF robot!

Expert Inconsistency: Humans operators might use different '**modes**' across trials



Recap: Multimodal Behavior

- Deterministic MSE BC policy will fail due to “**averaging**” of **modes**!
- **Goal**: learn the **full distribution**!



Generative Modeling

- **Goal:** Learn complex, **full distributions**

$$p_{\theta}(x) \rightarrow \text{in robotics } \pi_{\theta}(a_t | s_t)$$

How to Encode a
Distribution

VAEs

How to Sample from a
Distribution

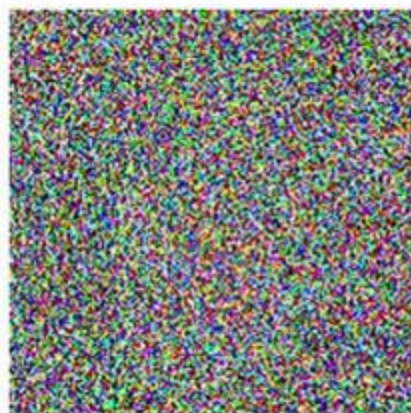
Diffusion Models

How to Transport
through a Distribution

Flow Models

Generative Modeling

- A generative model converts samples from an initial distribution $p(z)$ to a data distribution $p_{data}(x)$



Generative
Model



Latent Variable Models

- Output still gaussian, but receives additional input
- Can represent “any” distribution
- Popular: conditional VAEs

$$\pi(\mathbf{a} \mid \mathbf{o}, \mathbf{z}) = f_{\theta}(\mathbf{o}, \mathbf{z})$$

Conditional Decoder

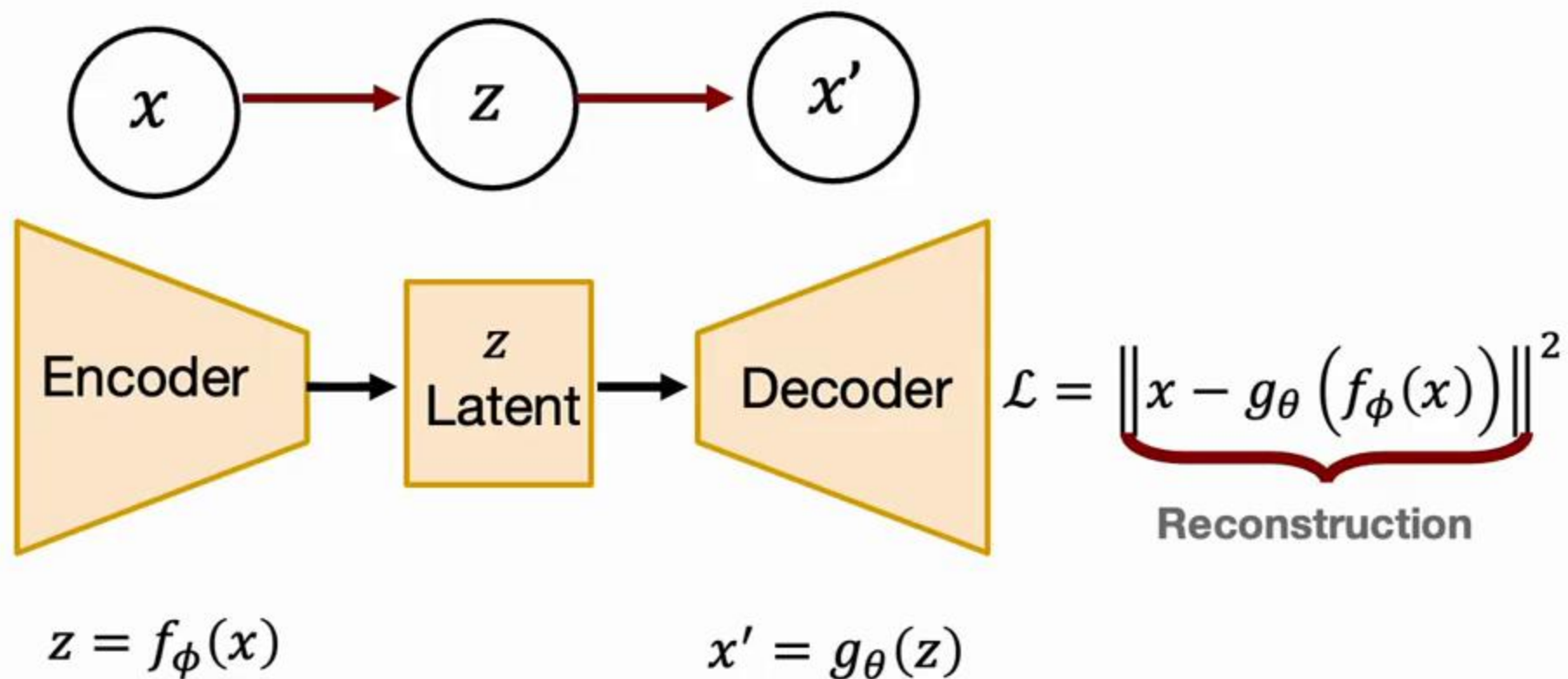


$z_{\text{left}} \sim \mathcal{N}(0, \mathbf{I})$

$z_{\text{right}} \sim \mathcal{N}(0, \mathbf{I})$

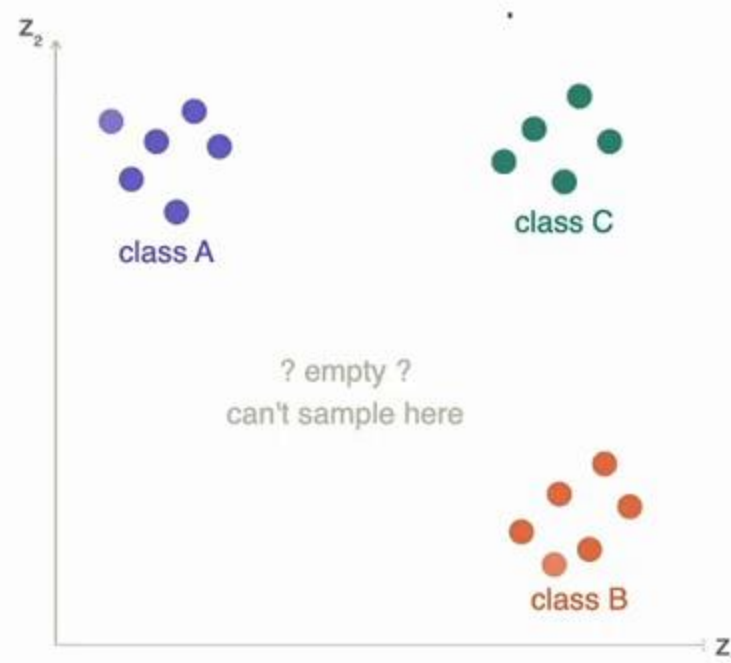
Autoencoder

- Compress input x into a compact latent representation z , then reconstruct x'



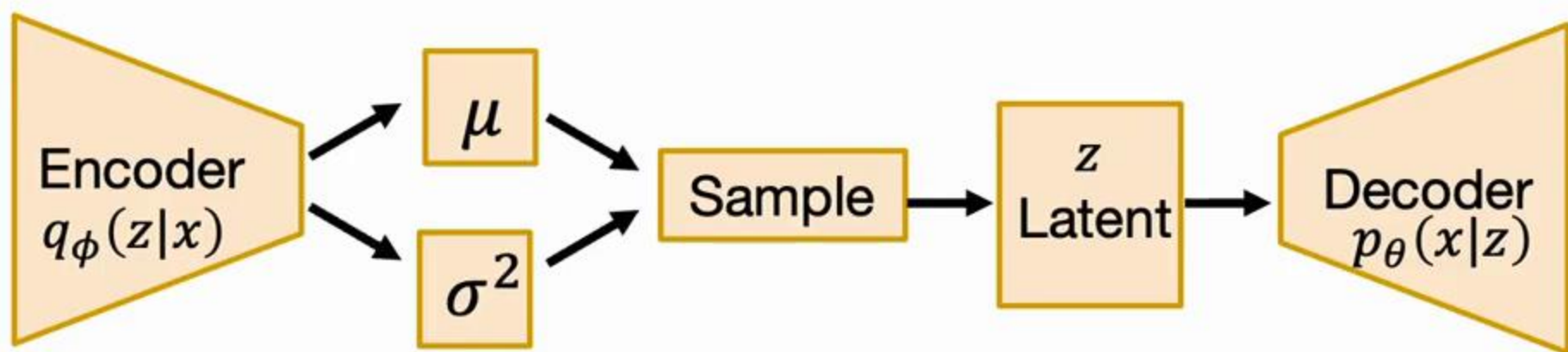
Issues with Autoencoders

- Latent space has no structure
- Bottleneck (size) is the only regularizer
- z is deterministic, can't sample new data!



Variational Autoencoders (VAEs)

- Constrain z to follow a **distribution**, often $p(z) = \mathcal{N}(0, I)$
 - Forces latent space to be continuous, structured & sampleable



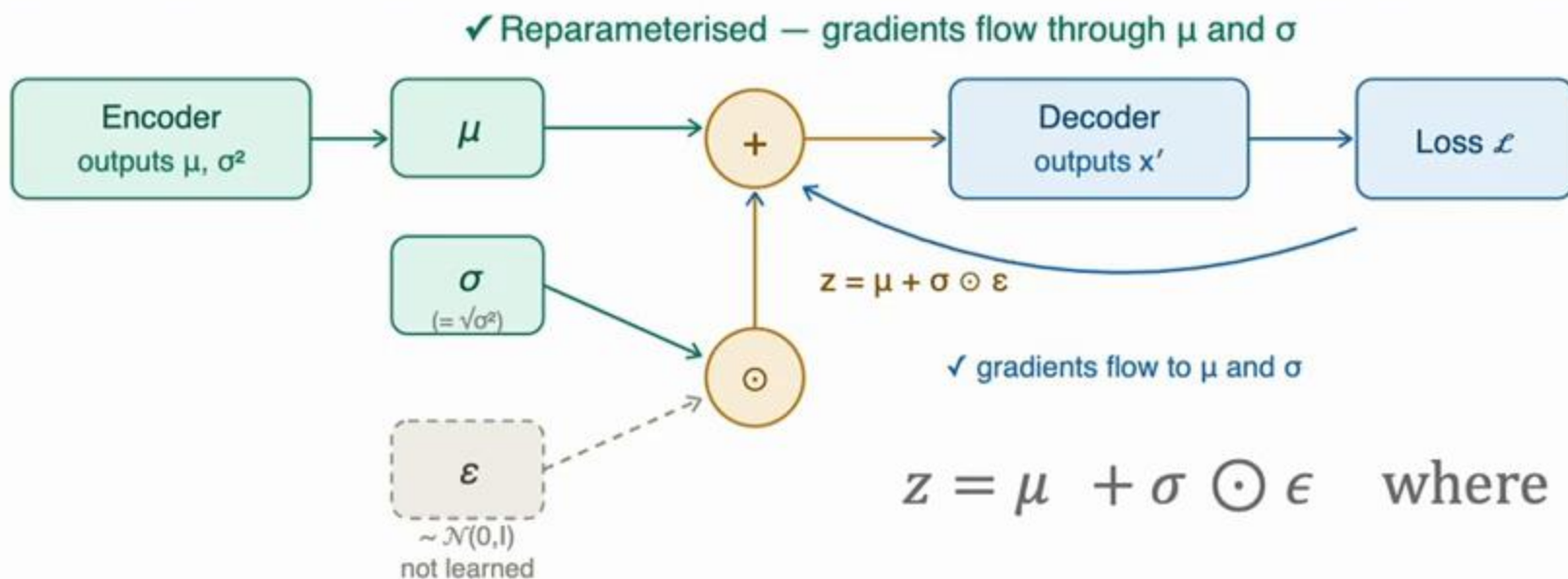
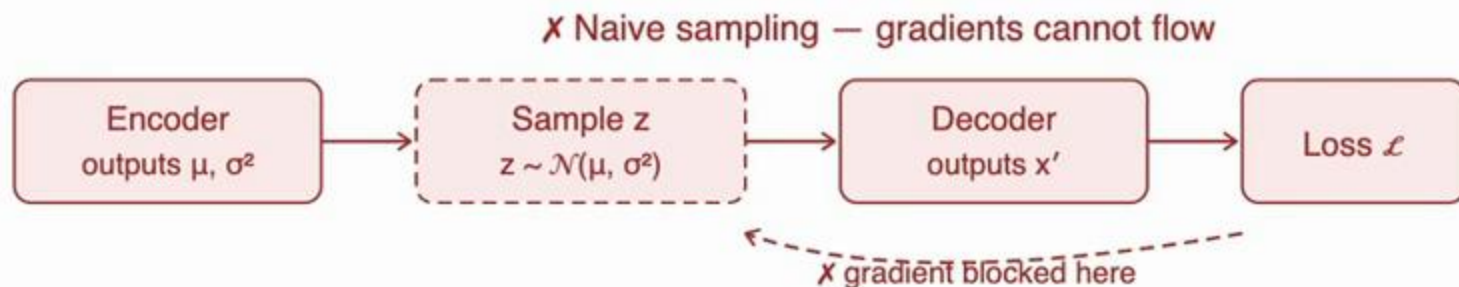
$$\log p_{\theta}(x) \geq \underbrace{-KL\left(q_{\phi}(z|x) \parallel p(z)\right)}_{\text{Regularization:}} + \underbrace{\mathbb{E}_{q_{\phi}(z|x)} [\log p_{\theta}(x|z)]}_{\text{Reconstruction:}}$$

Regularization:
are latents z close to our prior
 $p(z)$?

Reconstruction:
how well does latent z explain x ?

Reparameterization Trick

- Problem: Sampling is non-differentiable



Derivation

$$\nabla_{\phi} \mathcal{L} = \nabla_{\phi} \int q_{\phi}(z|x) f(z) dz$$

Option 1) Score Function Estimator - REINFORCE

$$\nabla_{\phi} \mathcal{L} = \mathbb{E}_{z \sim q_{\phi}(z|x)} [f(z) \nabla_{\phi} \log q_{\phi}(z|x)] \quad \rightarrow \quad \begin{array}{l} \nabla_{\phi} \text{ acts on } \log q_{\phi} \text{ only} \\ \text{High variance, as } \frac{\partial z}{\partial \phi} = 0 \end{array}$$

Derivation

$$\nabla_{\phi} \mathcal{L} = \nabla_{\phi} \int q_{\phi}(z|x) f(z) dz$$

Option 1) Score Function Estimator - REINFORCE

$$\nabla_{\phi} \mathcal{L} = \mathbb{E}_{z \sim q_{\phi}(z|x)} [f(z) \nabla_{\phi} \log q_{\phi}(z|x)] \quad \rightarrow \quad \begin{array}{l} \nabla_{\phi} \text{ acts on } \log q_{\phi} \text{ only} \\ \text{High variance, as } \frac{\partial z}{\partial \phi} = 0 \end{array}$$

Option 2) Reparameterization

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad \text{Reparam. } z \rightarrow \epsilon, p(\epsilon) \text{ independent of } \phi$$

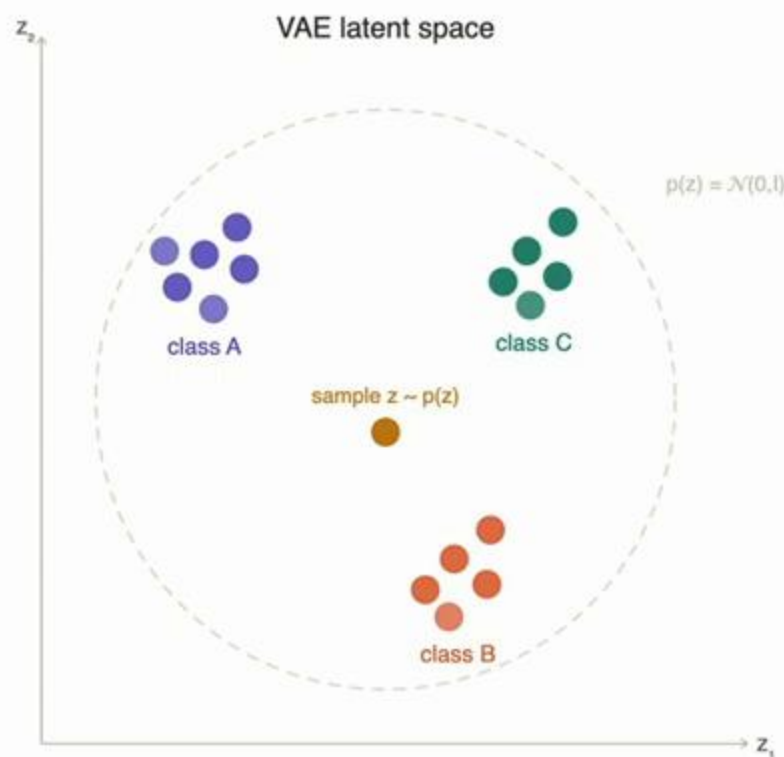
$$\nabla_{\phi} \mathcal{L} = \nabla_{\phi} \int p(\epsilon) f(\mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon) d\epsilon \quad \text{Leibniz/linearity, move grad. inside integral}$$

$$\nabla_{\phi} \mathcal{L} = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\nabla_{\phi} f(\mu_{\phi}(x) + \sigma_{\phi}(x) \odot \epsilon)] \quad \nabla_{\phi} \text{ acts on } f \text{ directly} \rightarrow \text{low variance } \frac{\partial z}{\partial \phi} \neq 0$$

$$\nabla_{\phi} \mathcal{L} \approx \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \phi}, \text{ where } \frac{\partial z}{\partial \mu_{\phi}} = 1, \quad \frac{\partial z}{\partial \sigma_{\phi}} = \epsilon \quad \text{Chain Rule}$$

Variational Autoencoders (VAEs)

- Latent space is continuous & structured
- Generative! Any $z \sim p(z)$ decodes meaningfully
- Smooth! Nearby z 's produce similar outputs



Issues with Variational Autoencoders

- **Posterior Collapse:** powerful decoder can learn to model $p(x)$ directly, while ignoring z
- **Prior Mismatch:** sampling $z \sim \mathcal{N}(0, \mathbf{I})$ at test time can land in regions the decoder was never trained on

Test time: $z \sim \mathcal{N}(0, \mathbf{I}) \rightarrow$ lands in z_{left} (✓), z_{right} (✓), or gap (✗)



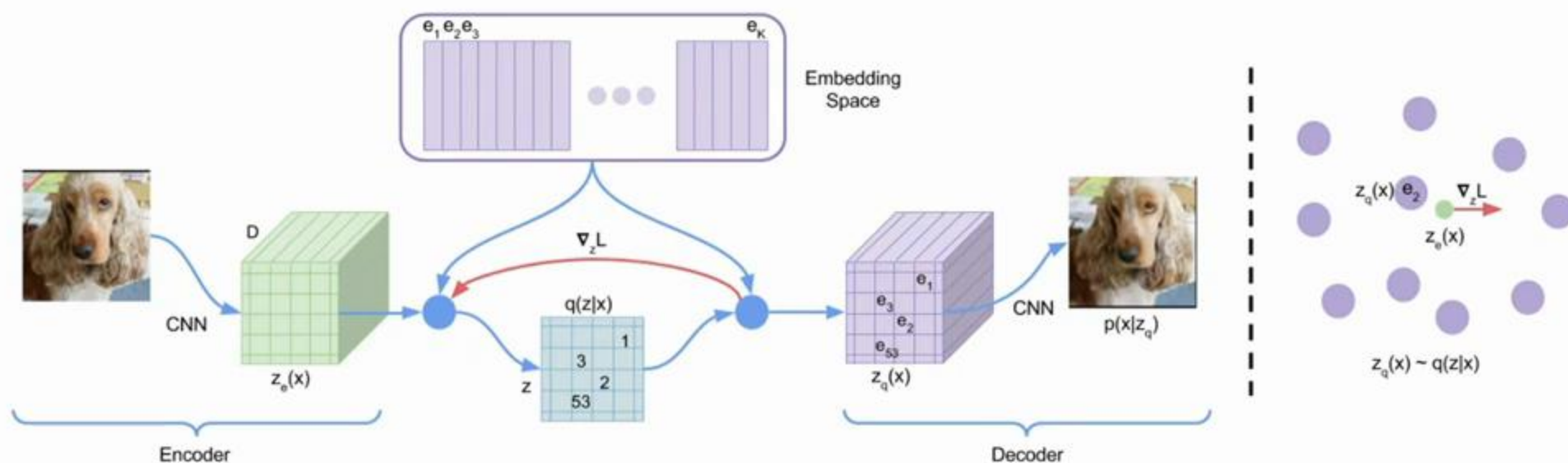
Training time:

$$z_{\text{left}} \sim q_{\phi}(z|x_{\text{left}}) = \mathcal{N}(\mu_{\text{left}}, \sigma_{\text{left}}^2)$$

$$z_{\text{right}} \sim q_{\phi}(z|x_{\text{right}}) = \mathcal{N}(\mu_{\text{right}}, \sigma_{\text{right}}^2)$$

Vector Quantized VAE (VQ-VAE)

- Replace the continuous Gaussian latent z with a **discrete codebook** of K learned vectors
- Force to “commit” to a discrete code



Neural Discrete Representation Learning
van den Ord, Vinyals, et al., (2017)

VQ-VAE

- What's the nearest codebook vector $\mathcal{C} = \{e_1, e_2, \dots, e_K\}$?

$$z_q(x) = e_k \quad \text{where} \quad k = \arg \min_j \|z_e(x) - e_j\|_2$$



Not Differentiable!

updating $z_e(x)$ will still point to same neighbor

Straight-Through Estimator

copy the gradients from the decoder input (z_q) directly to the encoder output (z_e), bypassing the argmin

$$\frac{\partial z_q}{\partial z_e} \approx \mathbf{I} \quad \Rightarrow \quad \nabla_{z_e} \mathcal{L} \approx \nabla_{z_q} \mathcal{L}$$

$$z_q = z_e + \text{sg}[z_q - z_e]$$

$$\frac{\partial z}{\partial z_e} = \frac{\partial z_e}{\partial z_e} + \frac{\partial \text{sg}[z_q - z_e]}{\partial z_e} = 1 + 0 = 1$$

VQ-VAE: Training

Full Loss

$$\mathcal{L} = \underbrace{\|x - D(z_q)\|_2^2}_{\text{Reconstruction}} + \underbrace{\|sg(z_e) - e_k\|_2^2}_{\text{Codebook}} + \underbrace{\beta \|z_e - sg(e_k)\|_2^2}_{\text{Commitment}}$$

Reconstruction

Did the decoder recover x from the quantised code?
Trains encoder & decoder jointly via STE

Codebook

Moves the codebook vectors e_k toward the encoder outputs to learn a representative vocabulary of the data

Commitment

Forces the encoder to stay close to its chosen codebook entry, preventing the latent space from growing arbitrarily

Training gives us the codebook $\mathcal{C}$ and decoder D ,
but **how do we sample new data at test time?**

VQ-VAE: How to Generate New Data?

1. Training Stage:

$$\underset{\text{encoder}}{x} \rightarrow z_e \xrightarrow{\arg \min} e_k \in \mathcal{C} = \{e_1, e_2, \dots, e_K\}$$

2. Collect code indices per training sample

$$\mathcal{D} = \{k^{(1)}, k^{(2)}, \dots, k^{(N)}\}$$

3. Learn a categorical prior over the sequence of codes:

$$p(k_1, \dots, k_n) = \prod_{i=1}^n p(k_i | k_1, \dots, k_{i-1}) \quad \text{i.e. autoregressive transformer...}$$

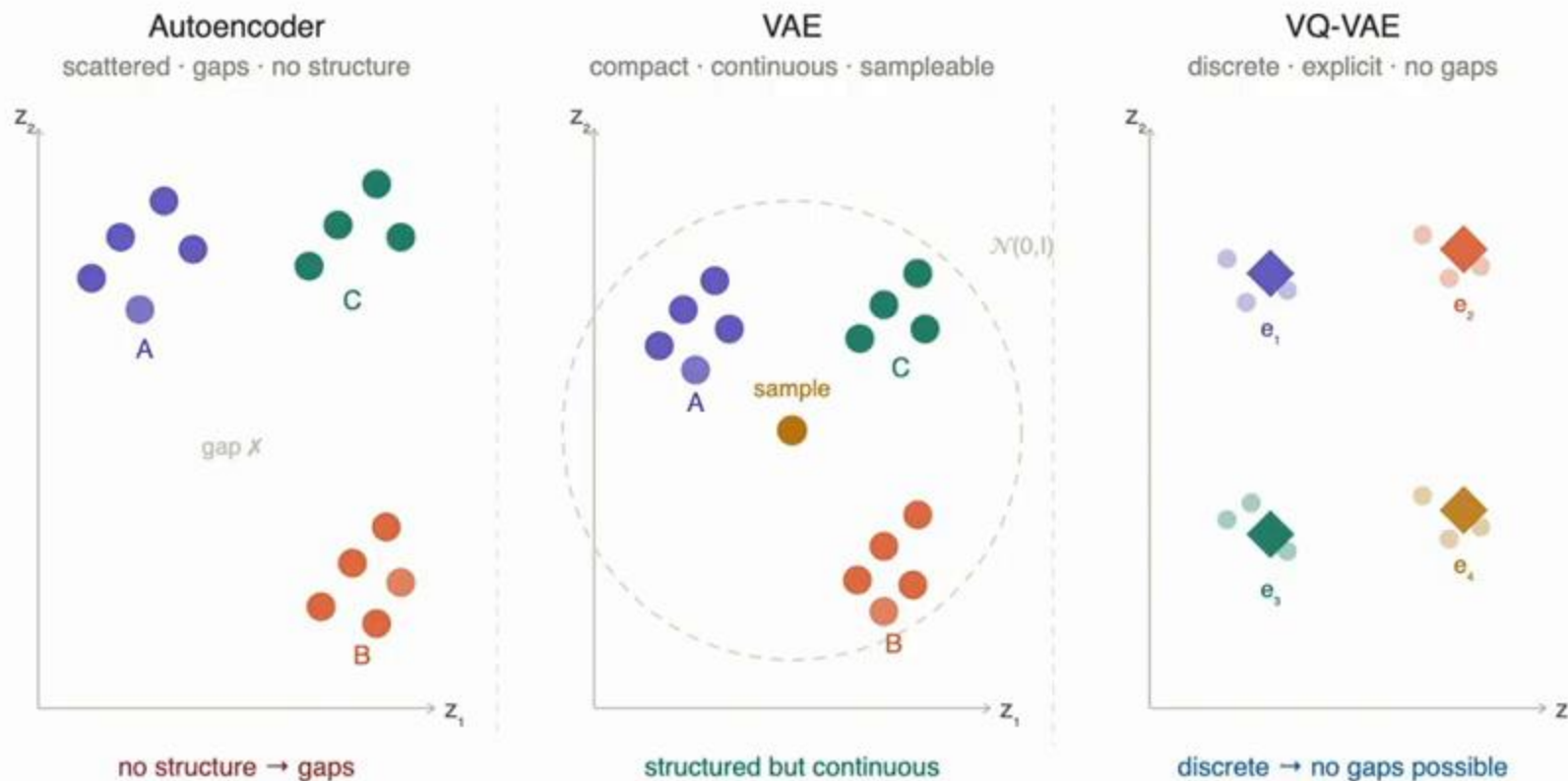
4. Inference time:

I) Sample Index: $k_i \sim p(k_i | k_1, \dots, k_{i-1})$

II) Lookup Vector: $e_{k_i} = \text{Lookup}(\mathcal{C}, k_i)$

III) Decode: $\hat{x} = D(e_{k_1}, \dots, e_{k_n})$

VQ-VAE

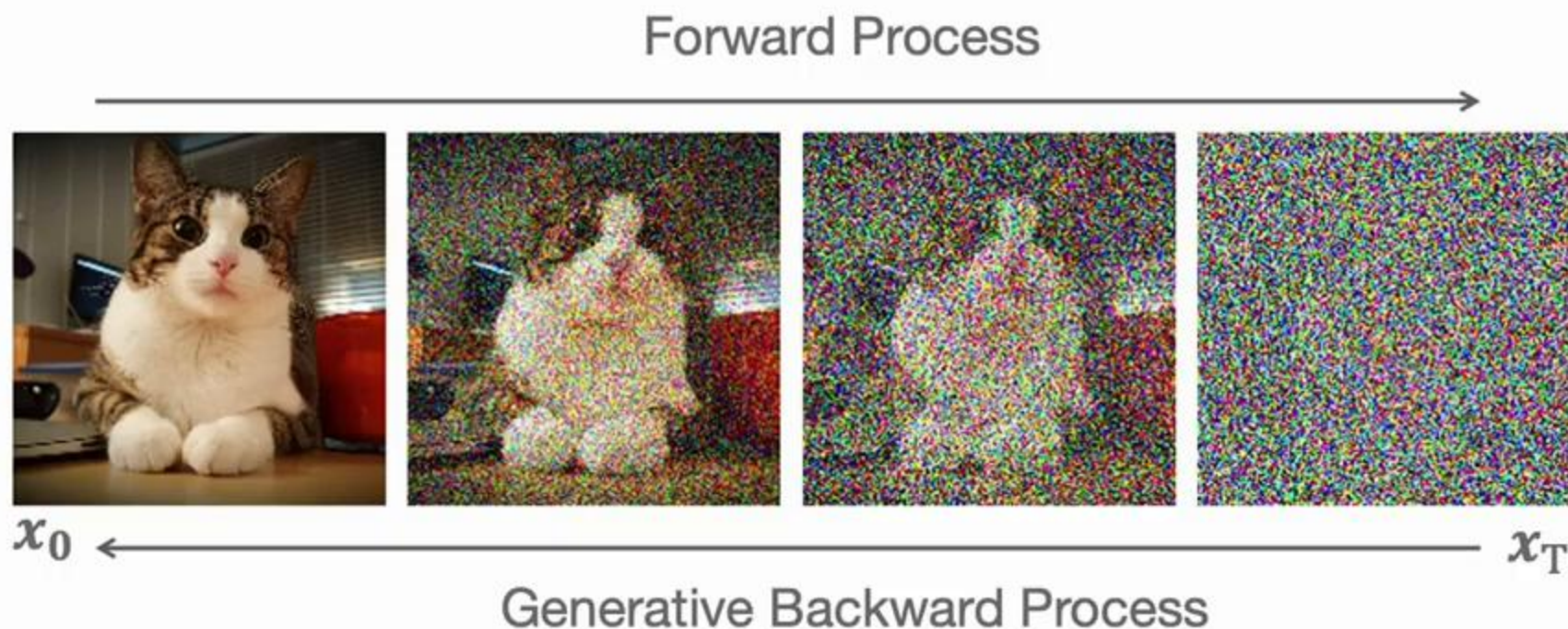


- ✓ Fixes posterior collapse
- ✓ No prior mismatch
- ✓ Discrete and controllable, good compression

- ✗ Codebook collapse → Finite Scalar Quantization (FSQ)
- ✗ Need to determine K in advance
- ✗ Reconstruction still uses MSE, mean-seeking in outputs remains

Diffusion

- VQ-VAEs still have mean-seeking in outputs due to MSE
- Diffusion learns complex distributions over continuous variables



Diffusion: Forward Process

- Iteratively add gaussian noise

$$x_{i+1} = \underbrace{\sqrt{1 - \beta_{i+1}}}_{\text{Signal Decay}} x_i + \underbrace{\sqrt{\beta_{i+1}}}_{\text{Noise Scale}} \epsilon_i$$

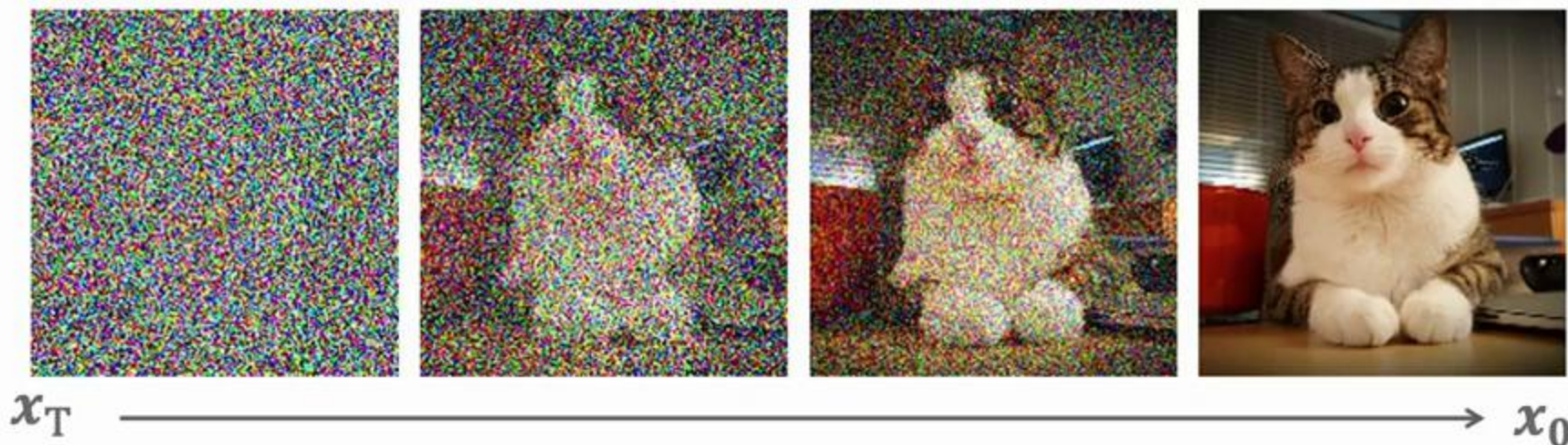
$\epsilon_i \sim \mathcal{N}(0, \mathbf{I})$ β_i : noise schedule

Reparametrization Trick!



Diffusion: Backward Process

- Learn $p(x_{i-1}|x_i)$, which is intractable (infinite clean images could have produced the noisy version)
- **Solution:** instead of predicting the previous image, predict the added noise $x_{i-1} \approx x_i - \epsilon_{\theta}(x_i, i)$!



Diffusion: Objective

- Same ELBO as VAE, but T denoising steps instead of one, and q is fixed

$$\log p_{\theta}(x_0) \geq \underbrace{-D_{KL}(q(x_T|x_0) \parallel p(x_T))}_{\text{Prior Regularization}} + \sum_{i=2}^T \underbrace{-D_{KL}(q(x_{i-1}|x_i, x_0) \parallel p_{\theta}(x_{i-1}|x_i))}_{\text{Denoising Matching}} + \underbrace{\log p_{\theta}(x_0|x_1)}_{\text{Reconstruction}}$$

- KL between two Gaussians has closed form

$$D_{KL} \propto \|\tilde{\mu}_i - \mu_{\theta}(x_i, i)\|^2 \propto \|\epsilon - \epsilon_{\theta}(x_i, i)\|^2$$

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{i \sim U(1, T), \epsilon \sim \mathcal{N}(0, \mathbf{I})} [\|\epsilon - \epsilon_{\theta}(x_i, i)\|^2]$$

Diffusion: Efficient Training

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{i \sim U(1,T), \epsilon \sim \mathcal{N}(0, I)} [\|\epsilon - \epsilon_{\theta}(x_i, i)\|^2]$$



Requires “teleporting” instead of iterative forward process $O(T)$

Signal Retention

for algebraic convenience



Forward Process

$$x_{i+1} = \sqrt{1 - \beta_{i+1}}x_i + \sqrt{\beta_{i+1}}\epsilon_i \quad \epsilon_i \sim \mathcal{N}(0, I)$$

$$\alpha_i = 1 - \beta_i$$

Expand

$$x_1 = \sqrt{\alpha_1}x_0 + \sqrt{1 - \alpha_1}\epsilon_0 \quad x_2 = \sqrt{\alpha_2}x_1 + \sqrt{1 - \alpha_2}\epsilon_1$$

Substitute

$$x_2 = \sqrt{\alpha_2}(\sqrt{\alpha_1}x_0 + \sqrt{1 - \alpha_1}\epsilon_0) + \sqrt{1 - \alpha_2}\epsilon_1 = \sqrt{\alpha_1\alpha_2}x_0 + \underbrace{\sqrt{\alpha_2(1 - \alpha_1)}}_{\alpha_2(1 - \alpha_1)}\epsilon_0 + \sqrt{1 - \alpha_2}\epsilon_1$$

$$\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \alpha_1\alpha_2$$

Gaussian Sum

$$\mathcal{N}(0, \sigma_a^2 I) + \mathcal{N}(0, \sigma_b^2 I) = \mathcal{N}(0, (\sigma_a^2 + \sigma_b^2)I) \rightarrow x_2 = \sqrt{\alpha_1\alpha_2}x_0 + \sqrt{1 - \alpha_1\alpha_2}\bar{\epsilon}$$

Generalizing

$$\text{Signal Coefficient} = \sqrt{\alpha_i\alpha_{i-1} \dots \alpha_1} = \sqrt{\prod_{j=1}^i \alpha_j} = \sqrt{\bar{\alpha}_i}$$

$$x_i = \sqrt{\bar{\alpha}_i}x_0 + \sqrt{1 - \bar{\alpha}_i}\bar{\epsilon}$$

Allows “teleporting” from x_0 to any noisy x_i in a single step $O(1)$!

Diffusion: DDPM Sampling

- Very slow! → Image generation $T=1000$!
1. Sample Gaussian Noise $x_T \sim \mathcal{N}(0, I)$
 2. For $i = T, \dots, 1$:
 - I. Sample $z \sim \mathcal{N}(0, I)$ if $i > 1$, else $z = 0$
 - II. Denoise $x_{i-1} = \underbrace{\frac{1}{\sqrt{\alpha_i}} \left(x_i - \frac{1-\alpha_i}{\sqrt{1-\bar{\alpha}_i}} \epsilon_\theta(x_i, i) \right)}_{\text{Predicted Mean}} + \underbrace{\sigma_i z}_{\text{Stochastic (Langevin) Term}}$

Typically $\sigma_i = \sqrt{\beta_i}$

We add noise during denoising to increase sample diversity and avoid collapsing into a mean distribution
 3. Return x_0

- DDPM is a discrete version of a Stochastic Differential Equation (SDE)
- DDPM denoising = score step ($\nabla \log(p)$) towards higher density + Langevin noise

Diffusion: DDIM Sampling

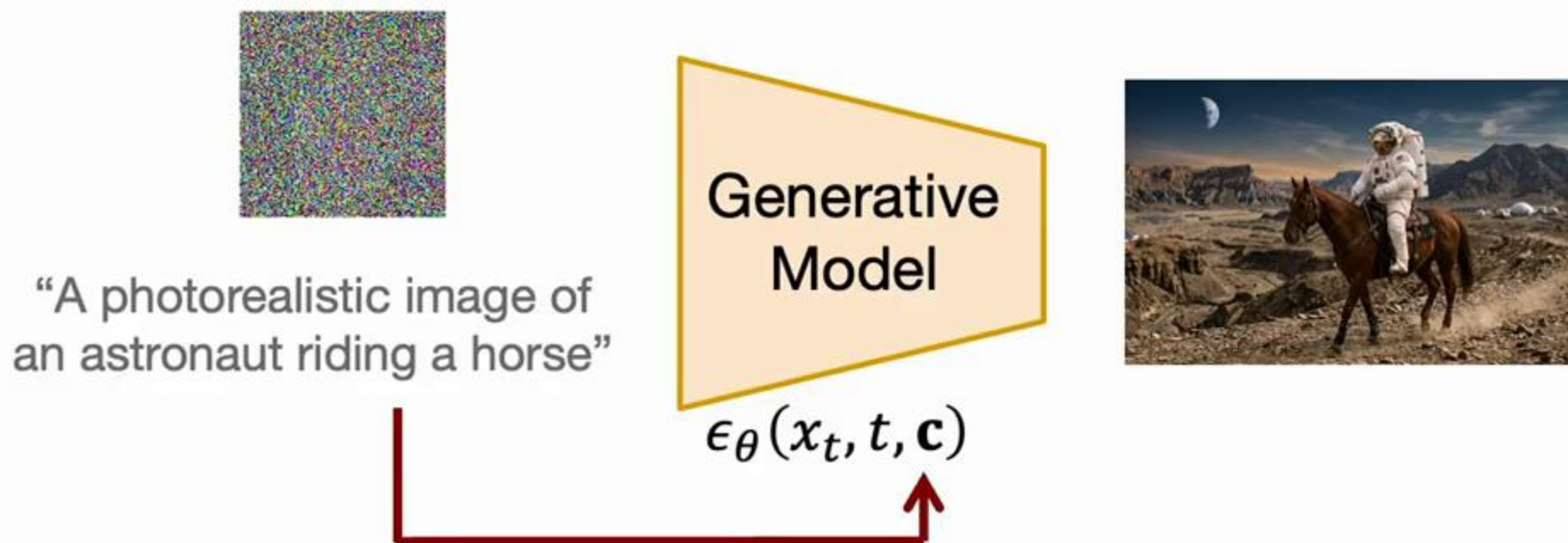
Denoising Diffusion Implicit Models
Song et al., (2020)

1. Sample Gaussian Noise $x_T \sim \mathcal{N}(0, I)$
2. Define subset of timesteps $\tau = \{t_1, t_2, \dots, t_K\}$ (only 20 steps instead of 1000)
3. For $i = K, K - 1, \dots, 1$: (where $x_i, \bar{\alpha}_i$ denote x_{t_i} and $\bar{\alpha}_{t_i}$)
 - I. Denoise $\hat{x}_0 = \frac{x_i - \sqrt{1 - \bar{\alpha}_i} \epsilon_\theta(x_i, i)}{\sqrt{\bar{\alpha}_i}}$  Inverse of “teleportation” formula, can jump to any $x_{i-1} \in \tau$ directly
 - II. Deterministic Re-Noise $x_{i-1} = \sqrt{\bar{\alpha}_{i-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{i-1}} \epsilon_\theta(x_i, i)$
 α_{i-1} from τ , not necessarily the previous timestep, this enables skipping timesteps
4. Return x_0  No noise term!

- Decouples number of denoising iterations in training & inference
- DDPM is markovian, DDIM defines a non-Markovian forward process that results in the exact same marginal distributions as DDPM
- DDIM discretizes the probability flow ODE

Everything covered so far is
unconditional generation from noise!

What about conditional generation?



Classifier-Free Guidance (CFG)

- **Problems:** Model might ignore conditioning, classifier guidance required training separate classifier $p(c|x_t)$
- **Solution:** train a single model for conditional & unconditional generation by randomly dropping the condition during training

Guidance Scale condition fidelity vs diversity trade-off

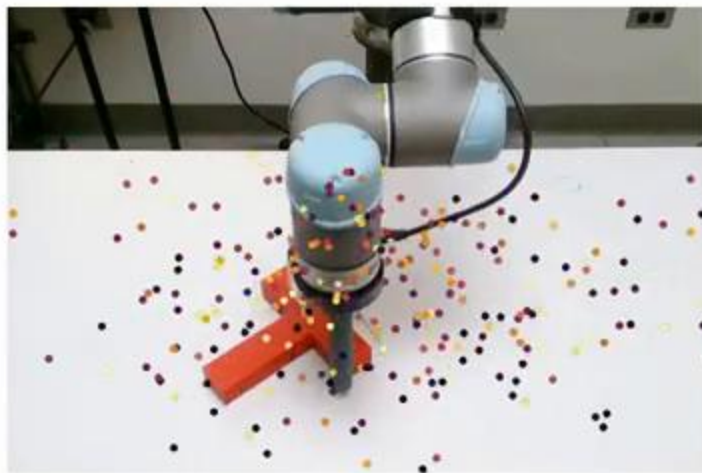
$$\hat{\epsilon} = \underbrace{\epsilon_{\theta}(x_t, t, \emptyset)}_{\text{Unconditional}} + w \underbrace{(\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \emptyset))}_{\text{Guidance=Conditional-Unconditional}}$$

Classifier-Free Diffusion Guidance
Ho & Salimans, (2022)

Case Study: Diffusion Policy

- **Key:** Denoises robot action sequences instead of images
- DDPM training, DDIM inference, injects observations into noise prediction network (no future state pred required)

$$\mathcal{L} = \text{MSE} \left(\epsilon^k, \epsilon_{\theta}(O_t, A_t^0 + \epsilon^k, k) \right) \quad A_t^{k-1} = \alpha \left(A_t^k - \gamma \epsilon_{\theta}(O_t, A_t^k, k) + \mathcal{N}(0, \sigma^2 I) \right)$$

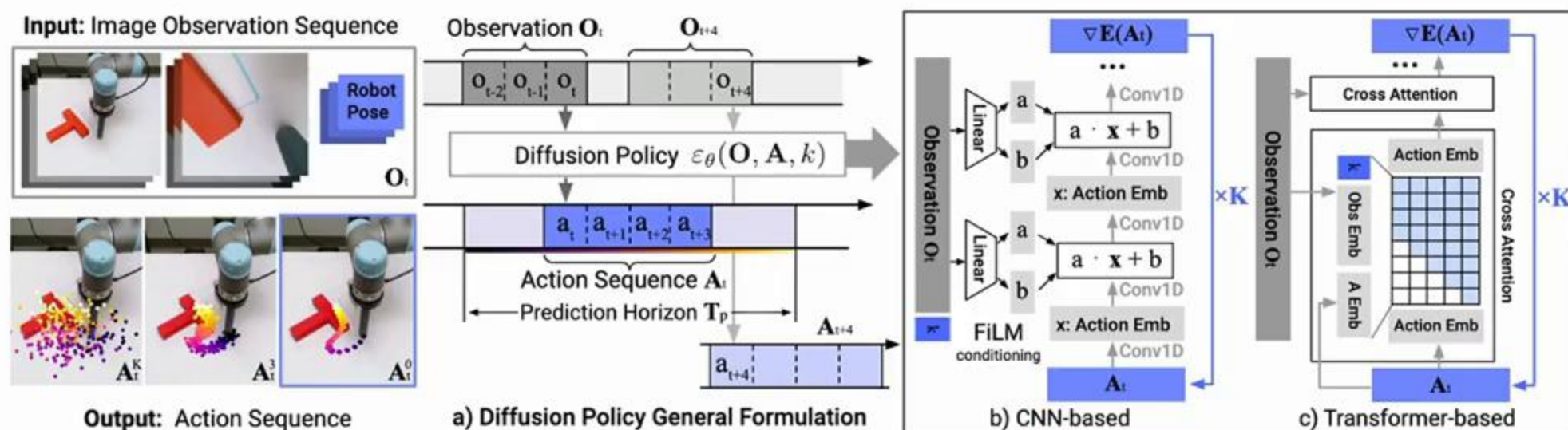


Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

Cheng Chi et al., (2023)

Case Study: Diffusion Policy

- O_t computed **once** before K denoising steps
- 1D temporal U-Net with O_t conditioned via Feature-wise Linear Modulation (FiLM)
- Handles multimodality in demonstrations!

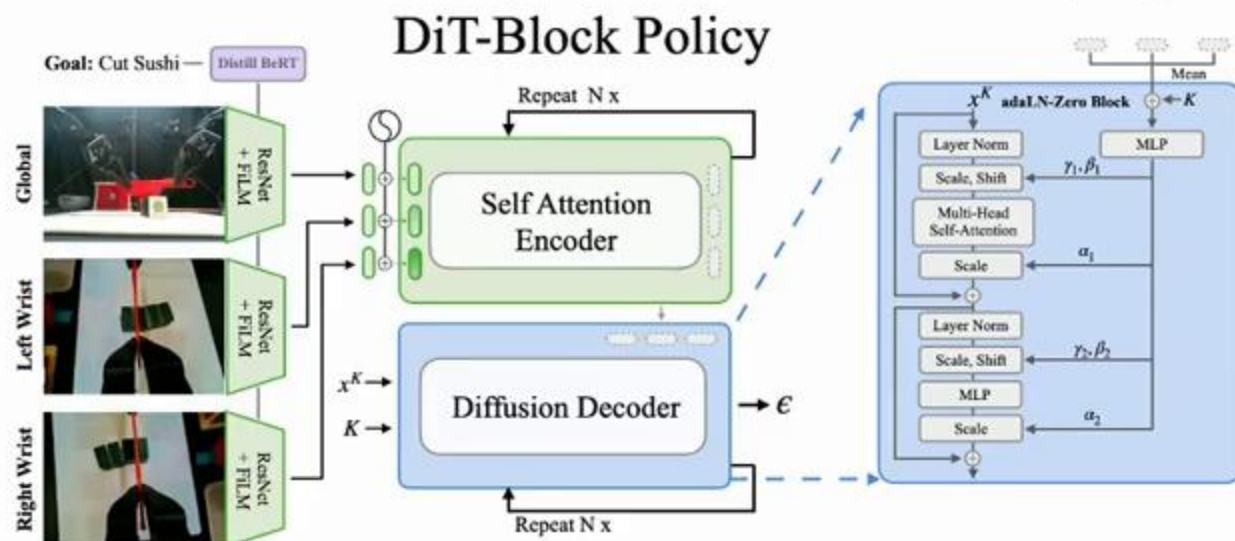
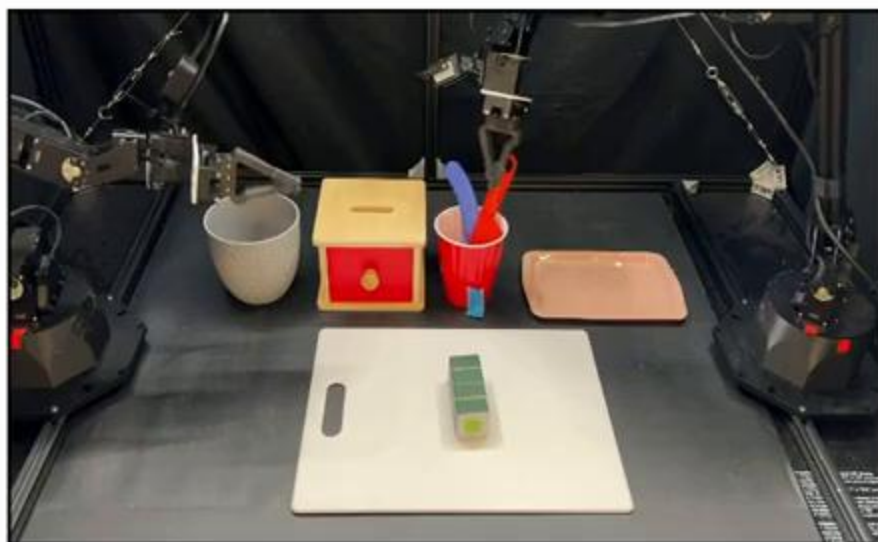


Diffusion Policy: Visuomotor Policy Learning via Action Diffusion

Cheng Chi et al., (2023)

Scaling Diffusion in Robotics: DiT Block Policy

- Replaces cross-attention with adaptive Layer-Norm
- Scales with Model & Data size
- Stable Training
- Long horizon bimanual dexterous tasks (1500+ timesteps)

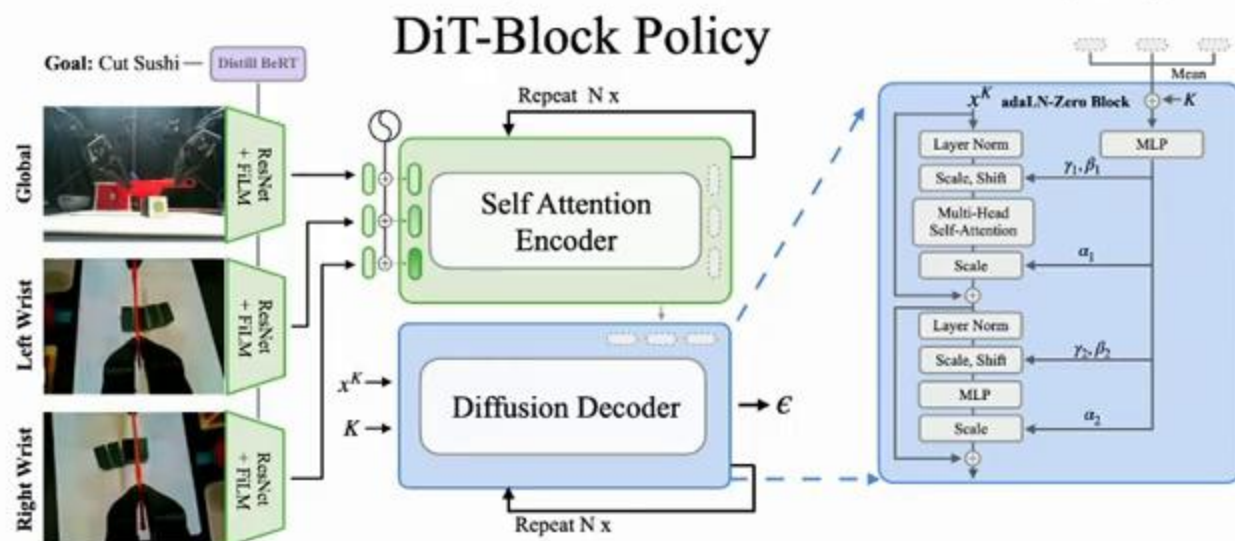
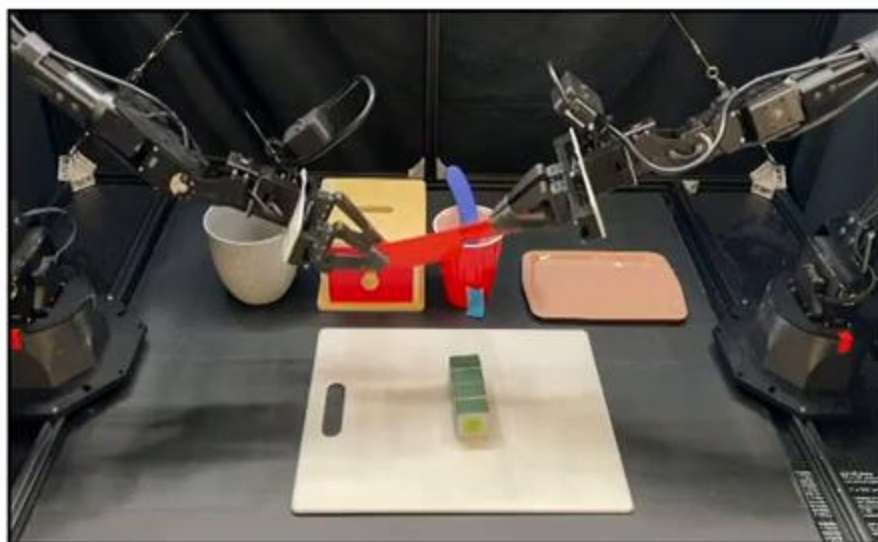


The Ingredients for Robotic Diffusion Transformers

Dasari, Mees et al., ICRA, 2025

Scaling Diffusion in Robotics: DiT Block Policy

- Replaces cross-attention with adaptive Layer-Norm
- Scales with Model & Data size
- Stable Training
- Long horizon bimanual dexterous tasks (1500+ timesteps)

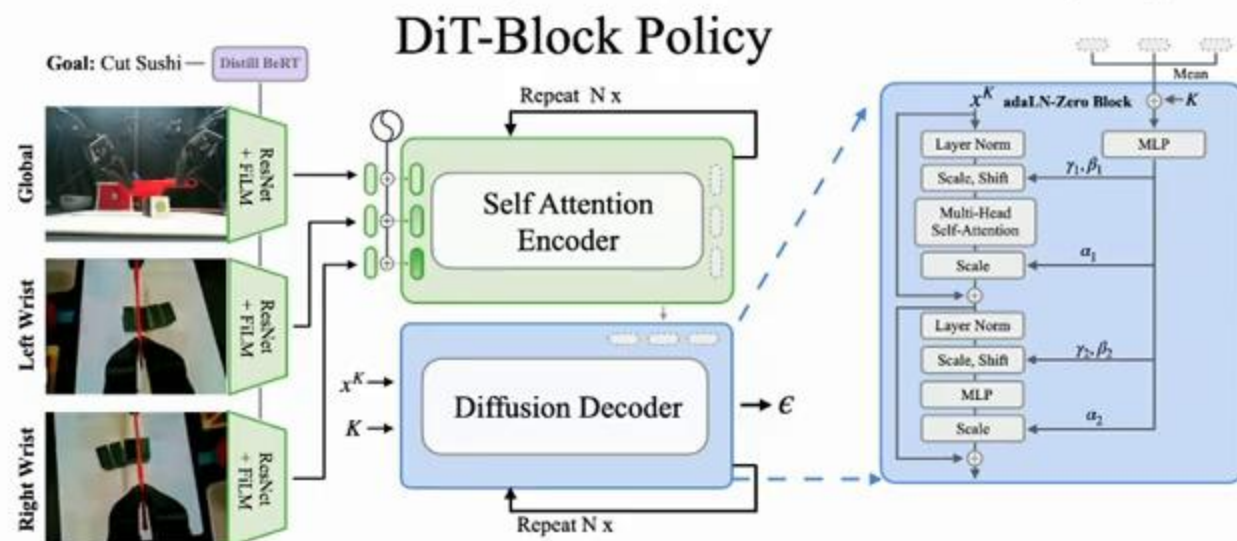
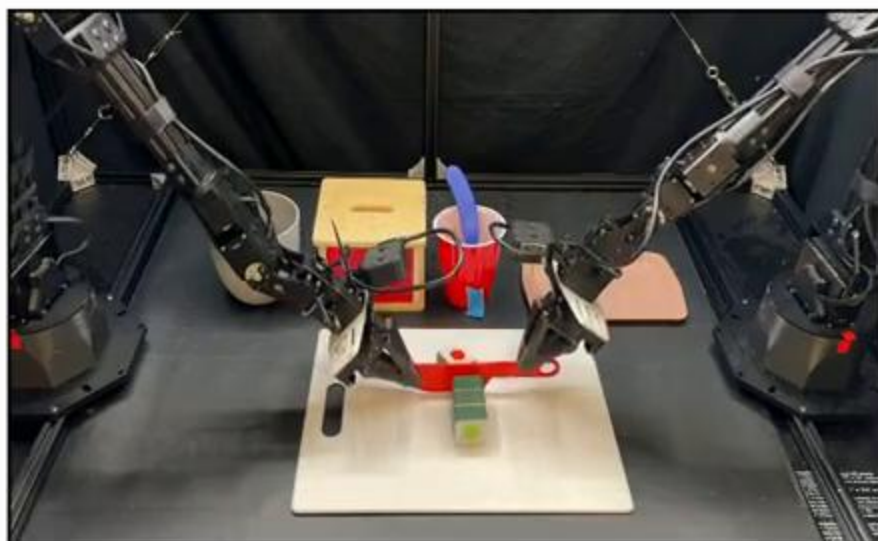


The Ingredients for Robotic Diffusion Transformers

Dasari, Mees et al., ICRA, 2025

Scaling Diffusion in Robotics: DiT Block Policy

- Replaces cross-attention with adaptive Layer-Norm
- Scales with Model & Data size
- Stable Training
- Long horizon bimanual dexterous tasks (1500+ timesteps)



The Ingredients for Robotic Diffusion Transformers

Dasari, Mees et al., ICRA, 2025

Flow Matching

- Learn a velocity field $v_\theta(x_t, t)$ that transports samples from noise $\epsilon \sim \mathcal{N}(0, I)$ to data $x_0 \sim p(x_0)$ along continuous paths

$$x_t = \underbrace{(1 - t)\epsilon + tx_0}_{\text{Straight line from noise to } x_0, \text{ no schedule}}, \quad \epsilon \sim \mathcal{N}(0, I), \quad t \in [0, 1]$$

Straight line from noise to x_0 , no schedule

Training:

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\underbrace{\|v_\theta(x_t, t)\|}_{\text{Predicted velocity}} - \underbrace{\|x_0 - \epsilon\|}_{\text{Ground-truth velocity from noise to } x_0 \text{ is constant, independent of } t}]^2$$

Predicted velocity Ground-truth velocity from noise to x_0 is constant, independent of t

Inference:

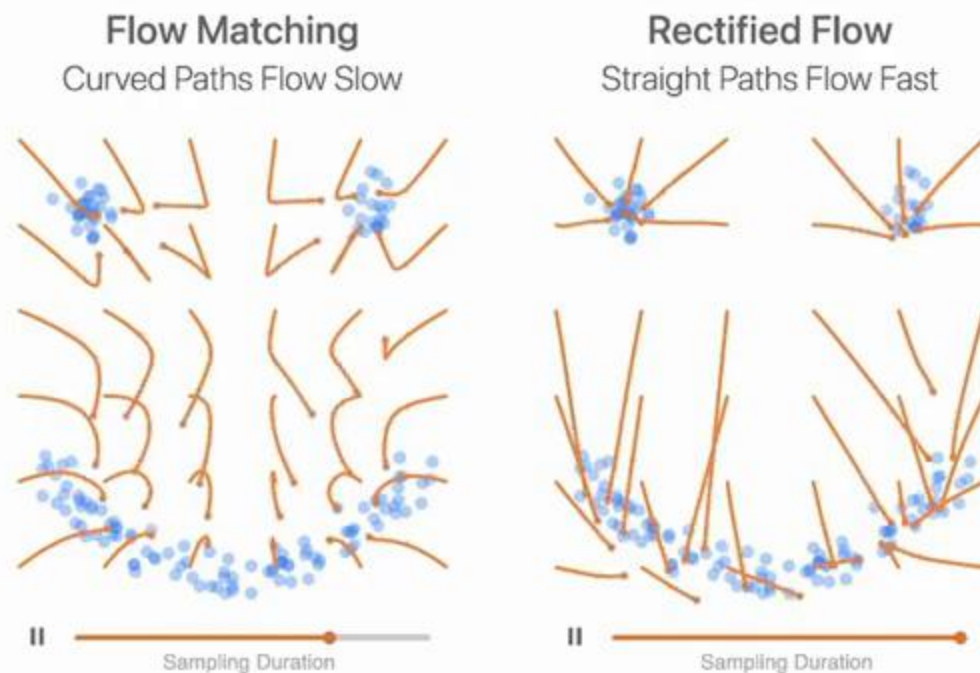
$$x_{t+\Delta t} = x_t + v_\theta(x_t, t)\Delta t$$

Integrate learned velocity with Euler ODE

$$\underbrace{\mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]}_{\text{Diffusion, predict noise}} \xleftrightarrow[\text{linear schedule}]{\text{With schedule}} \underbrace{\mathbb{E}[\|v_\theta(x_t, t) - (x_0 - \epsilon)\|^2]}_{\text{Flow Matching, predict velocity}}$$

Rectified Flow

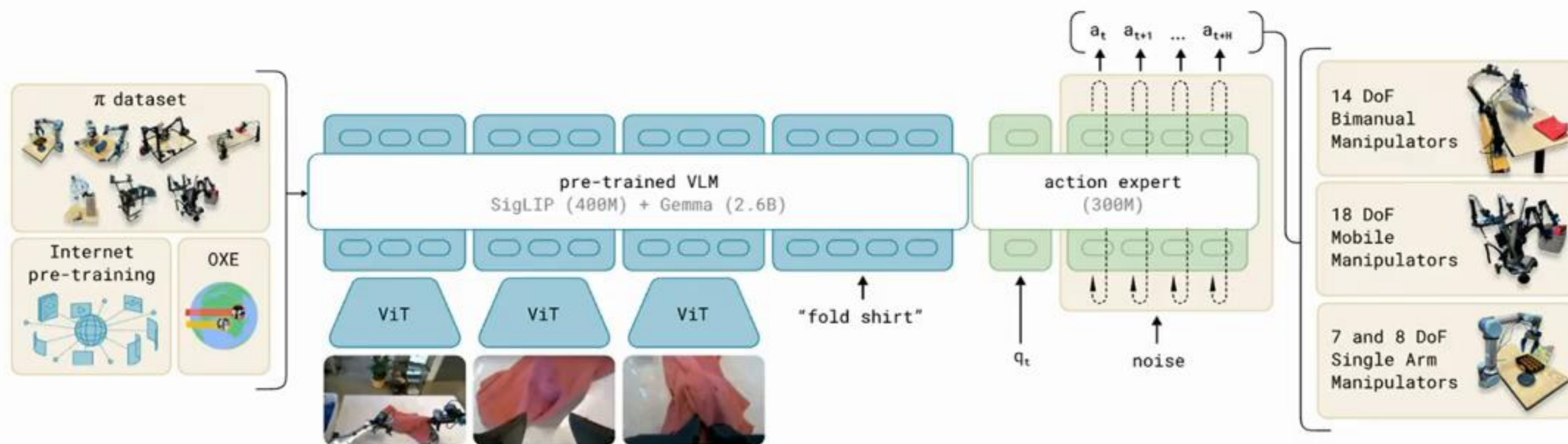
- Retrains Flow Matching on noise-data pairs generated by its own flow, untangling trajectory crossings & straightening paths



Source: Alec Helbling

Case Study: π_0

- Finetunes a VLM to produce actions via Flow Matching
- Samples flow-time τ from a shifted Beta distribution to focus on noisier (harder action prediction) parts of flow



π_0 : A Vision-Language-Action Flow Model for General Robot Control
Physical Intelligence (2024)

Conclusion

Latent Variable Models

AE: compressed representation via reconstruction

VAE: constrain latent to follow a distribution via reparametrization trick

VQ-VAE: Discretize latent space using learned codebook vectors

Diffusion Models

DDPM: Iteratively denoise Gaussian noise to generate samples

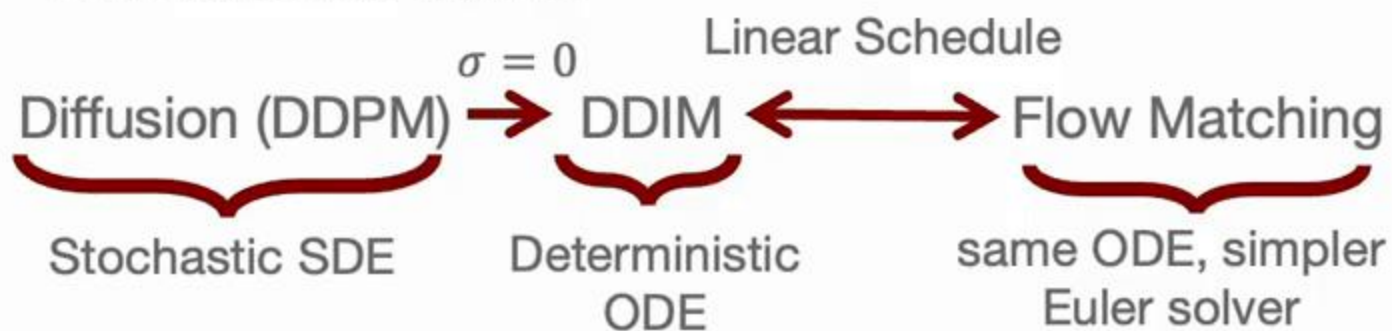
DDIM: Faster deterministic sampling via non-Markovian diffusion process

CFG: Steer generation by blending conditional & unconditional scores

Flow Models

Flow Matching: Learns velocity field to transport noise

Rectified Flow: Straighten ODE trajectories via iterative reflow for faster, fewer-step sampling



Thank you for your attention